Project Deliverables Part 2 - Progress Report: Architecture Finalization & Initial Implementation

Members: An Hong Pham, Jason Chu, Devansh Chaudhary

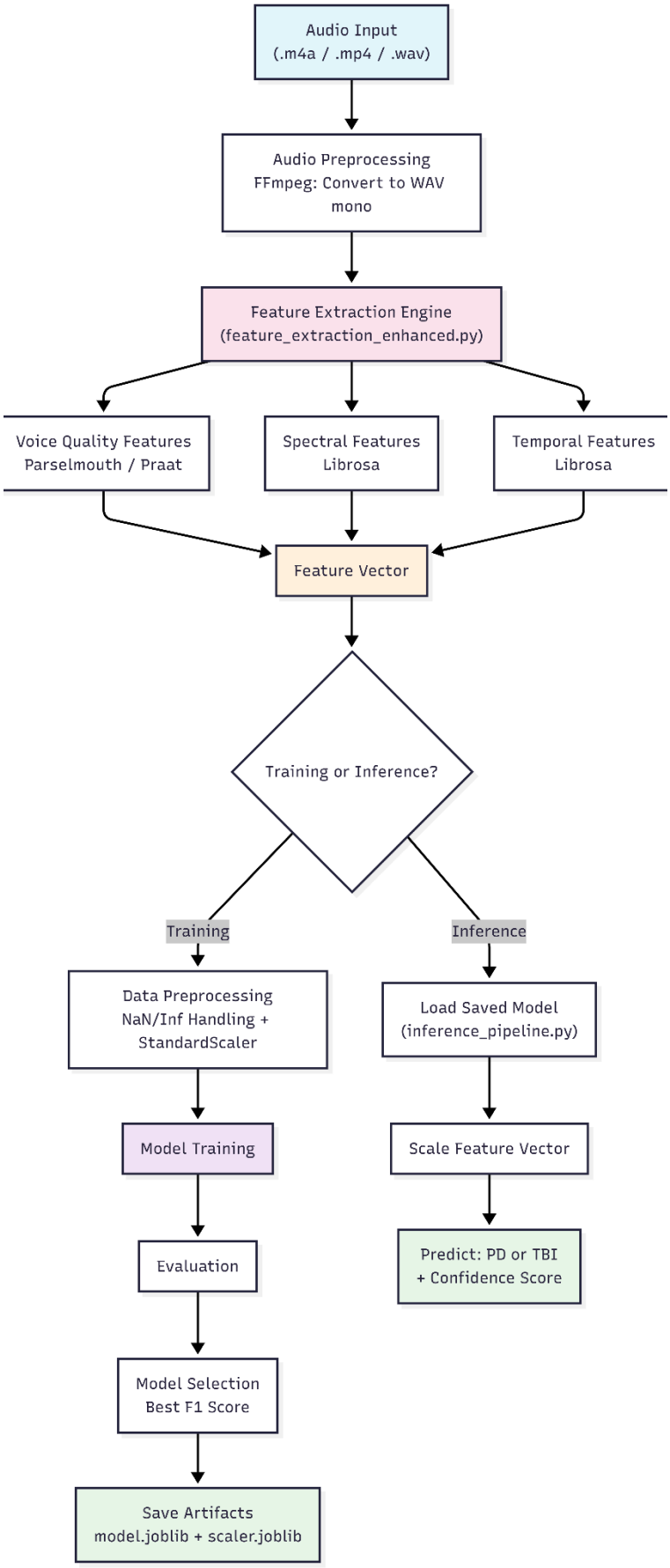
1. Finalized Architectural Flow & System Design

A. System Architecture Overview

Our system is a speech-based ML classification pipeline that distinguishes between Parkinson's Disease (PD) and Traumatic Brain Injury (TBI) patients using acoustic features extracted from speech recordings. The components included in this project:

- ML/AI Pipeline: Full feature extraction -> training -> inference pipeline
- Data Ingestion Pipeline: Batch processing of audio folders (PD/TBI)
- External Libraries: Librosa, Parselmouth (Praat) and other helper Python libraries

The comprehensive flow overview is on the next page.


Audio Input
(.m4a / .mp4 / .wav)
Audio Preprocessing
FFmpeg: Convert to WAV
mono
Feature Extraction Engine
(feature_extraction_enhanced.py)
Voice Quality Features
Parselmouth / Praat
Spectral Features
Librosa
Temporal Features
Librosa
Feature Vector
Training or Inference?
Training
Inference
Data Preprocessing
NaN/Inf Handling +
StandardScaler
Model Training
Evaluation
Model Selection
Best F1 Score
Save Artifacts
model.joblib + scaler.joblib
Load Saved Model
(inference_pipeline.py)
Scale Feature Vector
Predict: PD or TBI
+ Confidence Score

B. Component-Level Design Artifacts

The data ingestion pipeline handles audio file loading, format conversion, feature extraction, and CSV generation

I. Data ingestion pipeline: implemented

Flow: Audio Files → Format Check → WAV Conversion (if .m4a/.mp4/.mp3) → Dual Loading [librosa.load() for spectral/temporal + parselmouth.Sound() for voice quality] → Extract Features → Append Label → Repeat for All Files → DataFrame → Save to csv file

II. Model interaction flow: in progress

1. User calls predict("file_name.m4a")
2. Pipeline loads model artifacts (.joblib files)
3. Feature extractor processes audio → n-dimension feature dictionary
4. StandardScaler (python scikit) normalizes the feature vector
5. Model predicts class label (PD or TBI)
6. Model returns class probabilities [P(PD), P(TBI)]
7. User receives: "PD" or "TBI" + Confidence %

III. Feature extraction engine: in progress

- Spectral: Librosa
- Temporal: Librosa
- Voice quality: Parselmouth (Praat)

IV. Training vs inference architecture: not started

2. Technology Stack (Implemented & In Use)

ML / AI Components

- Model selected:
  - Tree based: Random Forest, XGBoost, Decision Tree, Stochastic Gradient Descent and Gradient Boosting Classifier
  - Linear models: SVM, LDA, Logistic Regression

- Hosting strategy: Jupyter Notebook for prototyping and testing, saved as .joblib files, and inference by standalone Python script
- Integration method: audio files from USF Box, local features extraction using Librosa and Praat, and model artifacts serialized with joblibs.

3. Implementation Progress Summary

Status update:

- Completed: audio datasets collecting, feature extractions (54 features)
- Under development: ML training pipeline and inference pipeline
- Remains to be implemented: hyperparameter tuning, full model training once the datasets are available.

Screenshots:

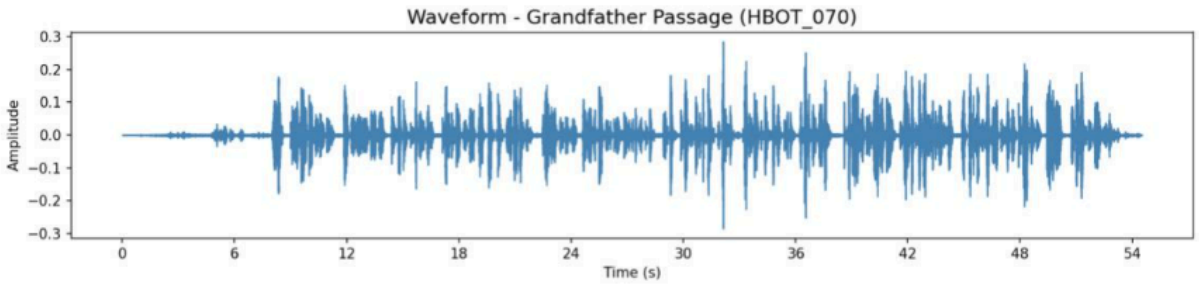


Figure 1: Waveform of the Grandfather Passage recording (54.5 seconds). Visible speech segments and pauses.

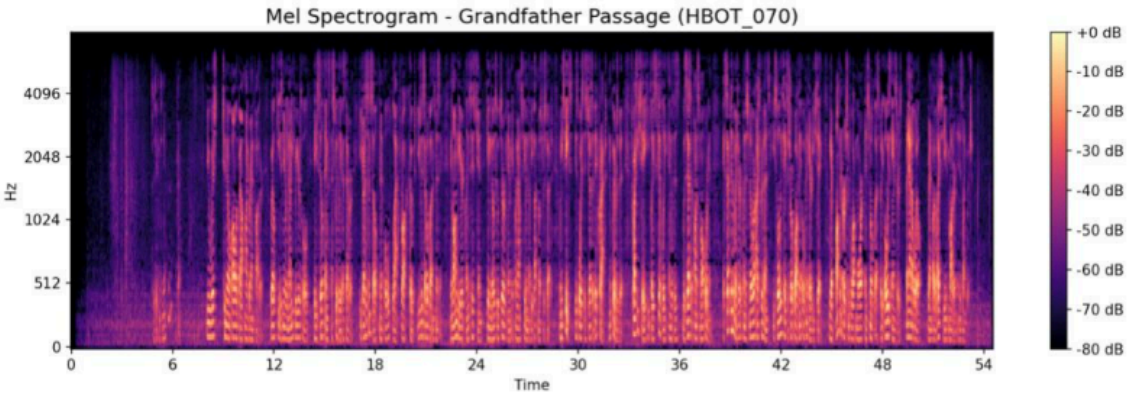


Figure 2: Mel spectrogram showing frequency content over time. Darker vertical bands indicate pauses. Energy concentration in lower frequencies typical for male speech.

4. Updated Implementation Roadmap

Required Milestones + Updated Timeline:

Milestones	Status	Date
Initial code (23 features)	Completed	Mar 9th - 13th
Feature Extraction	Completed	Mar 16th - 19th
ML training pipeline	In progress	Mar 23th - 28th
Inference	Planned	Mar 23th - 28th
Full model training	Planned	Apr 21st - 24th
Final model selection + Report	Planned	Apr 30th - May 3rd

Individual Contributions:

- An: Report writing, models research, features extraction
- Devansh: Python scripts for graphing, features extraction
- Jason: Report Writing, Features Extraction

Technical risks:

Risk	Severity	Mitigation
Audio quality variation	Medium	Standardize to mono WAV and standardize
NaN/inf in features	Medium	Replace with median values, skip corrupted data